

Introduction to Flutter

Flutter is an open-source UI software development kit created by Google. It is used for building natively compiled applications for mobile, web, and desktop from a single codebase. Flutter utilizes the Dart programming language and its rendering engine is written in C/C++.

Features of Flutter

- **Cross-Platform Development:**

Build applications for multiple platforms (iOS, Android, web, desktop) from a single codebase.

- **Hot Reload:**

Instantly see the effects of code changes during development without restarting the application.

- **Rich UI with Customizable Widgets:**

Offers a comprehensive set of pre-built, customizable widgets to create expressive and visually appealing user interfaces.

- **Fast Performance:**

Compiles to native code, resulting in high-performance applications with smooth animations.

- **Access to Native Features:**

Provides plugins to seamlessly integrate with device-specific features like camera, GPS, and storage.

- **Dart Programming Language:**

Uses Dart, an object-oriented programming language, for development.

- **Strong Community Support:**

Benefits from a growing and active developer community and extensive documentation.

Advantages of Flutter

- **Faster Development:** Hot Reload and a single codebase significantly reduce development time.
- **Cost-Effective:** Developing for multiple platforms with one codebase lowers development costs.

- **Expressive and Flexible UI:** Rich widget set and customization options allow for unique and engaging user interfaces.
- **Near-Native Performance:** Compiling to native code ensures high performance and a smooth user experience.
- **Strong Backing by Google:** Ensures continuous updates, improvements, and long-term support.

Disadvantages of Flutter

- **Learning Curve for Dart:**

Developers unfamiliar with Dart may need time to learn the language.

- **Larger App Size:**

Flutter applications can sometimes have a larger file size compared to native apps due to the framework and rendering engine being bundled.

- **Limited Third-Party Libraries:**

While growing, the ecosystem of third-party libraries and packages is still smaller compared to more established frameworks.

- **Potential for Platform-Specific Code:**

In some complex scenarios requiring deep native integration, platform channels might be needed, requiring platform-specific code.

Here is a very basic "Hello World" Flutter application that can be created and run in Android Studio:

1. Project Setup in Android Studio:

- Open Android Studio.
- Go to `File > New > New Flutter Project`.
- Select `Flutter Application` and click `Next`.
- Provide a `Project name` (e.g., `hello_world_app`), verify your `Flutter SDK Path`, choose a `Project location`, and add a `Description`. Click `Next`.
- Configure the `Company domain` (e.g., `com.example.helloworld`) and click `Finish`.

2. The `main.dart` Code:

Android Studio will generate a basic project structure including a `lib/main.dart` file. Replace its content with the following:

Code

```
import 'package:flutter/material.dart';
```

```

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Hello World App',
      theme: ThemeData(
        primarySwatch: Colors.blue,
      ),
      home: Scaffold(
        appBar: AppBar(
          title: const Text('Hello World'),
        ),
        body: const Center(
          child: Text(
            'Hello, Flutter!',
            style: TextStyle(fontSize: 24),
          ),
        ),
      ),
    );
  }
}

```

3. Explanation of the Code:

- `import 'package:flutter/material.dart';`

This line imports the Flutter Material Design library, which provides pre-built widgets and styling for creating visually appealing UIs.

- `void main() { runApp(const MyApp()); }`

This is the entry point of the Flutter application. `runApp()` takes a `Widget` as an argument, which is the root of your widget tree. Here, it's `MyApp`.

- `class MyApp extends StatelessWidget:`

This defines a custom widget named `MyApp`. `StatelessWidget` means this widget's state (its data) doesn't change over time.

- `const MyApp({super.key});`

This is the constructor for `MyApp`. `super.key` is used for widget identification in the widget tree.

- `@override Widget build(BuildContext context):`

Every `StatelessWidget` (and `StatefulWidget`) must implement the `build` method. This method describes the UI of the

`widget`. `BuildContext` provides information about the widget's location in the widget tree.

- `MaterialApp(...)`:

This is the root widget for a Material Design application. It provides essential features like navigation, theming, and title.

- `title: 'Hello World App'`: Sets the title of the application (visible in the task switcher on Android).
- `theme: ThemeData(primarySwatch: Colors.blue)`: Defines the overall theme of the application, setting the primary color to blue.
- `home: Scaffold(...)`: The `home` property takes the widget that will be displayed as the main content of the app. `Scaffold` provides a basic visual structure for Material Design apps, including an `AppBar` and a `body`.
- `AppBar(...)`:

Displays a horizontal bar at the top of the screen.

- `title: const Text('Hello World')`: Sets the title text displayed in the `AppBar`.
- `body: const Center(...)`:
The main content area of the `Scaffold`.
- `Center(...)`: A widget that centers its child widget within its available space.
- `child: Text('Hello, Flutter!', style: TextStyle(fontSize: 24))`: The actual "Hello, Flutter!" text, styled with a font size of 24.

4. Running the Application:

- Select your target device (emulator or physical device) from the Android Studio toolbar.
- Click the green `Run` button (or `Shift + F10`).

Architecture of the Flutter framework

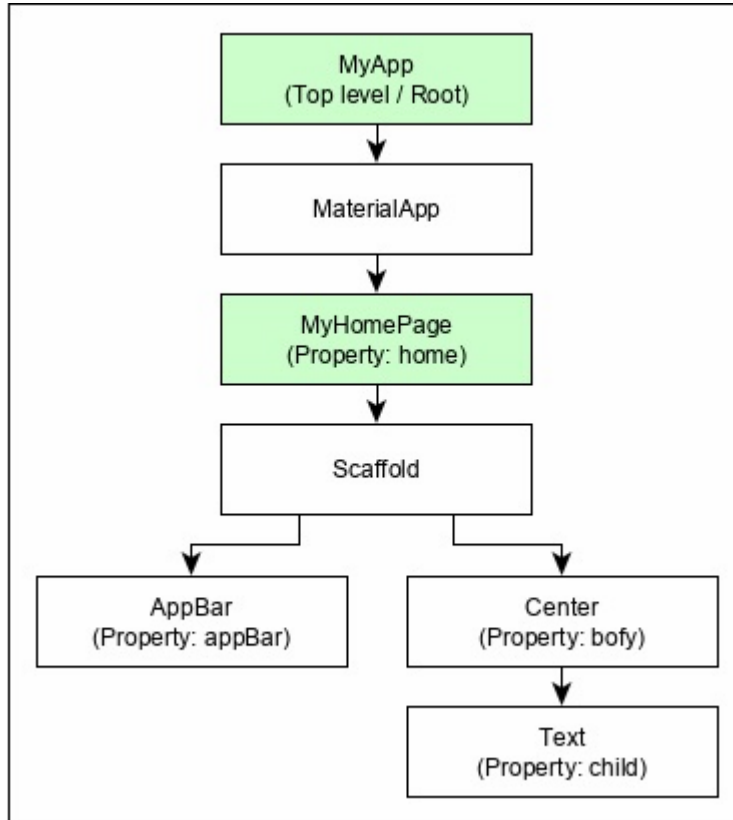
Widgets

The core concept of the Flutter framework is **In Flutter, Everything is a widget**. Widgets are basically user interface components used to create the user interface of the application.

In *Flutter*, the application is itself a widget. The application is the top-level widget and its UI is build using one or more children (widgets), which again

build using its children widgets. This **composability** feature helps us to create a user interface of any complexity.

For example, the widget hierarchy of the hello world application (created in previous example) is as specified in the following diagram –



Here the following points are worth notable –

- *MyApp* is the user created widget and it is build using the Flutter native widget, *MaterialApp*.
- *MaterialApp* has a home property to specify the user interface of the home page, which is again a user created widget, *MyHomePage*.
- *MyHomePage* is build using another flutter native widget, *Scaffold*
- *Scaffold* has two properties *body* and *appBar*
- *body* is used to specify its main user interface and *appBar* is used to specify its header user interface
- *Header UI* is build using flutter native widget, *AppBar* and *Body UI* is build using *Center* widget.
- The *Center* widget has a property, *Child*, which refers the actual content and it is build using *Text* widget

Gestures

Flutter widgets support interaction through a special widget, *GestureDetector*. *GestureDetector* is an invisible widget having the ability to capture user interactions such as tapping, dragging, etc., of its child

widget. Many native widgets of Flutter support interaction through the use of *GestureDetector*. We can also incorporate interactive feature into the existing widget by composing it with the *GestureDetector* widget. We will learn the gestures separately in the upcoming chapters.

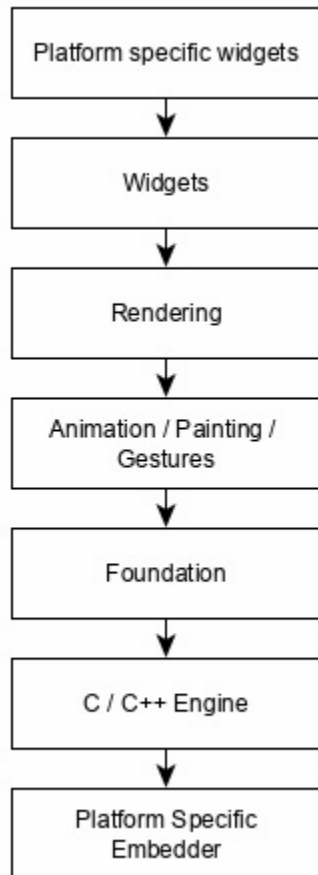
Concept of State

Flutter widgets support *State maintenance* by providing a special widget, *StatefulWidget*. Widget needs to be derived from *StatefulWidget* widget to support state maintenance and all other widget should be derived from *StatefulWidget*. Flutter widgets are **reactive** in nature. This is similar to reactjs and *StatefulWidget* will be auto re-rendered whenever its internal state is changed. The re-rendering is optimized by finding the difference between old and new widget UI and rendering only the necessary changes

Layers

The most important concept of Flutter framework is that the framework is grouped into multiple category in terms of complexity and clearly arranged in layers of decreasing complexity. A layer is build using its immediate next level layer. The top most layer is widget specific to *Android* and *iOS*. The next layer has all flutter native widgets. The next layer is *Rendering layer*, which is low level renderer component and renders everything in the flutter app. Layers goes down to core platform specific code

The general overview of a layer in Flutter is specified in the below diagram –



Develop a basic Flutter UI using widgets like Scaffold, AppBar, ListView, and FloatingActionButton.

To create a basic Flutter UI, you can use a `Scaffold` widget to provide a standard material design layout. The `Scaffold` includes properties for key UI elements, such as an `AppBar` at the top, a `body` for the main content, and a `FloatingActionButton` for a prominent action. The `ListView` widget is typically placed inside the `body` to display a scrollable list of items.

Basic Flutter UI example

This code creates a scrollable list of 50 items with an app bar and a floating action button. Tapping the button will display a message at the bottom of the screen using a `SnackBar`.

dart

```
import 'package:flutter/material.dart';

void main() {
  runApp(const MyApp());
}

class MyApp extends StatelessWidget {
  const MyApp({super.key});

  @override
  Widget build(BuildContext context) {
    return MaterialApp(
      title: 'Flutter Basic UI',
      theme: ThemeData(
        primarySwatch: Colors.blue,
        useMaterial3: true,
      ),
      home: const BasicScaffoldScreen(),
    );
  }
}

class BasicScaffoldScreen extends StatelessWidget {
  const BasicScaffoldScreen({super.key});

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: const Text('Flutter Basic UI'),
        backgroundColor: Theme.of(context).colorScheme.inversePrimary,
      ),
      body: ListView.builder(
        itemCount: 50,
        itemBuilder: (BuildContext context, int index) {
          return ListTile(
            leading: const Icon(Icons.star),
```

```

        title: Text('List Item ${index + 1}'),
      );
    },
  ),
  floatingActionButton: FloatingActionButton(
    onPressed: () {
      ScaffoldMessenger.of(context).showSnackBar(
        const SnackBar(
          content: Text('Floating Action Button Pressed!'),
        ),
      );
    },
    child: const Icon(Icons.add),
  ),
);
}
}
}
Use code with caution.

```

Explanation of the code

- `main()` function: The entry point of the app. It calls `runApp()` with a `MyApp` widget.
- `MyApp`: This is the root widget of the app.
 - `MaterialApp`: A convenient widget that wraps a number of widgets that are commonly required for material design applications, including a `Scaffold`.
- `BasicScaffoldScreen`: This widget contains the main screen layout.
 - `Scaffold`: Implements the basic material design visual layout structure. It provides an `AppBar`, `body`, and `FloatingActionButton`.
 - `AppBar`: A top bar with a title. The `title` property takes a `Text` widget, and the `backgroundColor` is set using the app's theme.
 - `body`: The main content of the screen.
 - `ListView.builder`: A more efficient way to create a scrollable list with a large number of items. It builds a child widget only when it scrolls into view, which saves memory.
 - `itemCount`: Defines the total number of items to display in the list.

- `itemBuilder`: A function that's called to build each item in the list. It returns a `ListTile` widget, which is a convenient way to create a row in a list.
- `FloatingActionButton`: A circular button that floats above the UI. It is used for a primary action.
- `onPressed`: A callback function that's executed when the button is tapped.
- `ScaffoldMessenger.of(context).showSnackBar()`: Displays a temporary message (`SnackBar`) at the bottom of the screen.
- `child`: The widget displayed inside the button, which is an `Icon` in this case.